

ЛАБОРАТОРНАЯ РАБОТА № 7

ПОСТРОЕНИЕ ПРОСТЫХ ПАРАМЕТРИЗОВАННЫХ ИЗОБРАЖЕНИЙ С ПОМОЩЬЮ ЛИСП-ПРОГРАММ

Цель работы: изучение способов задания геометрических точек в программе на *Visual LISP*, методики построения параметризованных изображений с помощью программных инструментов.

1. ТЕОРЕТИЧЕСКИЙ МАТЕРИАЛ

1.1. Геометрические точки в программе

Приведенные ранее простые примеры не демонстрируют все преимущества программирования на *Visual LISP* по сравнению с использованием графического редактора. Рассмотрим программу, иллюстрирующую принцип построения параметризованных изображений. Для такого построения требуется задавать геометрические объекты «внутри программы», т.е. без участия пользователя.

Геометрическая точка в программе может быть задана пятью способами.

Первый нам уже известен – ввод точки пользователем через функцию *GETPOINT*.

Второй – непосредственное указание координат точки в виде списка из двух (двухмерная точка) или трех (трехмерная точка) элементов с помощью функции *QUOTE*.

Примеры:

(QUOTE (1.0 1.0)) – точка с координатами 1,1; *'(1.0 1.0)* – аналогично;
'(1.0 2.0 3.0) – точка в трехмерном пространстве с координатами $X=1$, $Y=2$, $Z=3$.

Третий способ – задание точки через другую точку, угол и расстояние с помощью функции *POLAR*:

(POLAR <точка> <угол> <расстояние>).

Эта функция возвращает точку, находящуюся под углом *<угол>* и на расстоянии *<расстояние>* от заданной *<точки>*. Угол указывается в радианах против часовой стрелки.

POLAR – основная функция для геометрических построений внутри программы. Ее можно и удобно использовать практически во всех случаях.

Четвертый способ – определение точки пересечения двух отрезков с помощью функции *INTERS*:

(INTERS <точка1> <точка2> <точка3> <точка4> [<признак>]).

Функция *INTERS* находит точку пересечения двух отрезков – отрезка от *<точки1>* к *<точке2>* с отрезком от *<точки3>* к *<точке4>* и возвращает найденную точку; *<признак>* – любое выражение *Visual LISP*. Если *<признак>* равен *NIL*, то будет определена точка пересечения линий бесконечной длины, наложенных на заданные отрезки. Если *<признак>* имеет любое значение, отличное от *NIL*, или если аргумент *<признак>* вообще отсутствует, то ищется

точка пересечения только внутри отрезков. Если точка пересечения в этой ситуации не найдена, то функция *INTERSECT* возвращает *NIL*.

Пятый способ – с помощью функций *ANGLE* и *DISTANCE*.

Функция *ANGLE* задается в виде правила: (*ANGLE* <точка1> <точка2>).

Функция *ANGLE* возвращает действительное число, равное углу в радианах, образованному прямой, проходящей через <точка1> и <точка2>, с осью *X*.

Функция (*DISTANCE* <точка1> <точка2>) возвращает действительное число, равное расстоянию между <точкой1> и <точкой2>.

1.2. Принцип построения параметризованных изображений в программе

Принцип построения параметризованных изображений иллюстрирует программа 3, позволяющая построить квадрат с задаваемой стороной и вписанную в него окружность. Аналогично можно строить и более сложные изображения: для получения нового изображения с помощью программы пользователь задает новые значения параметров (точки, расстояния, числа и др.). Из этих данных в программе могут быть получены другие данные с помощью математических функций, функции *POLAR* и т.п. Полученные данные будут использованы в качестве аргументов команд графической системы посредством функции *COMMAND*.

```
; Программа 3. Квадрат с вписанной окружностью
(DEFUN C:QUADR ( ) ; заголовок описываемой функции
; Зададим начальную точку – левый нижний угол квадрата:
(SETQ P1 (GETPOINT "\nНач. точка:"))
; Зададим длину квадрата:
(SETQ L (GETDIST "\nДлина стороны:"))
; определим три оставшиеся вершины квадрата:
(SETQ P2 (POLAR P1 0.0 L))
(SETQ P3 (POLAR P2 (/ PI 2) L)) ; угол = PI/2
(SETQ P4 (POLAR P3 PI L)); угол = PI
; Построим квадрат на экране:
(COMMAND "LINE" P1 P2 P3 P4 "C")
; (Здесь "C" задает опцию "Замкнуть")
; Определим радиус окружности:
(SETQ R(/L 2)); радиус R=L/2
; Определим центр окружности PC:
(SETQ PR (POLAR P1 0.0 R)) ;PR - точка касания
; на нижней стороне
(SETQ PC (POLAR PR (/ PI 2) R))
; Построим окружность на экране:
(COMMAND "CIRCLE" PC R))
```

) ; Конец описания функции *QUADR*

Обратим внимание на несколько необычное имя функции – *C:QUADR*. Символы «*C:*» имеют определенный смысл, который будет разъяснен далее.

2. ЭТАПЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

- 2.1. Изучите способы задания геометрической точки.
- 2.2. Изучите принцип построения параметризованного изображения.
- 2.3. Изучите пример программы построения параметризованного изображения из лекции. Выберите два параметра для своей программы.
- 2.4. Напишите ЛИСП-программу вычерчивания геометрической фигуры в соответствии с индивидуальным заданием.
- 2.5. В среде *NanoCAD* проведите отладку написанной программы.

3. ПЛАН ОТЧЕТА

1. Титульный лист.
2. Цель работы.
3. Задание.
4. Описание программы.

ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ № 7

№ п/п	Фигура	
	вписанная	описанная
1	Правильный треугольник	Окружность
2	Окружность	Правильный треугольник
3	Квадрат	Окружность
4	Окружность	Правильный шестиугольник
5	Правильный шестиугольник	Окружность
6	Правильный пятиугольник	Окружность
7	Окружность	Правильный пятиугольник
8	Правильный семиугольник	Окружность
9	Окружность	Ромб
10	Ромб	Окружность
11	Окружность	Правильный семиугольник
12	Правильный восьмиугольник	Окружность
13	Окружность	Правильный восьмиугольник
14	Окружность	Равнобедренный непрямоугольный треугольник
15	Прямоугольный (неравнобедренный) треугольник	Окружность
15	Равнобедренный непрямоугольный треугольник	Окружность

16	Окружность	Прямоугольный (неравнобедренный) треугольник
17	Окружность	Равнобедренный прямоугольный треугольник
18	Равнобедренный прямоугольный треугольник	Окружность
19	Неравнобедренная трапеция	Окружность
20	Окружность	Неравнобедренная трапеция
21	Равнобедренная трапеция	Окружность
22	Окружность	Равнобедренная трапеция
23	Прямоугольная трапеция	Окружность
24	Окружность	Прямоугольная трапеция
25	Прямоугольник	Окружность
26	Окружность	Прямоугольник
27	Параллелограмм	Окружность
28	Окружность	Параллелограмм
29	Произвольный (неправильный) треугольник	Окружность
30	Окружность	Произвольный (неправильный) треугольник
31	Произвольный (неправильный) четырёхугольник	Окружность
32	Окружность	Произвольный (неправильный) четырёхугольник
33	Произвольный (неправильный) пятиугольник	Окружность
34	Окружность	Произвольный (неправильный) пятиугольник
35	Произвольный (неправильный) шестиугольник	Окружность